Team Members
Wayne Lu, Blessing Ajughu,
Jaqueline Lomas Vazquez, Anashrah Adil
Emaan Gohar

“Image Caption Generator”

AI-Based Automatic Image Description Generator

Technologies

PyTorch • Hugging Face • BLIP • Google Collab • COCO Dataset

The Problem:

Millions of images online do not include captions or descriptions. This creates accessibility problems for visually impaired users and makes large image collections difficult to organize.

Challenges:

- Manual captioning takes time
- Images without captions reduce accessibility
- Organizing large image datasets is difficult

Goals:

Create an AI system that automatically generates captions for uploaded images.

Our Solution:

We developed an AI image caption generator using a fine-tuned BLIP model that automatically creates captions from uploaded images.

System Features: Upload image, Analyze image content, Generate natural language captions, and Display multiple caption predictions

Example:
Input: Image of child riding bike
Output: “A photo showing a little boy riding a bike.”

Technical Approach:

Model Used:
BLIP Vision-Language Model

Frameworks & Tools:
Python, PyTorch, Hugging Face Transformers, and Google Collab

Workflow:

Image → Preprocessing → Vision Encoder → Decoder → Caption Output

Deployment:

Fine-tuned model uploaded to Hugging Face for live inference.

Figure 1.
Fine-tuned BLIP model successfully loaded from Hugging Face for inference testing.

✅ **Load trained model from Hugging Face**

Loads the fine-tuned model directly from Hugging Face.

```
MODEL_HF = "bajughu/group22600-image-caption-generator"

processor = BlipProcessor.from_pretrained(MODEL_HF)
model     = BlipForConditionalGeneration.from_pretrained(MODEL_HF).to(DEVICE)
model.eval()

print("✅ Fine-tuned model loaded from Hugging Face.")
print(f"   Model      : {MODEL_HF}")
print(f"   Parameters: {sum(p.numel() for p in model.parameters()):,}")
```

```
Loading weights: 100%                    473/473 [00:02<00:00, 248.60it/s, Materializing param=vision_model.post_layernorm.weight]
The tied weights mapping and config for this model specifies to tie text_decoder.cls.predictions.bias to text_decoder.cls.predictions.decoder.bias, but both are
The tied weights mapping and config for this model specifies to tie text_decoder.bert.embeddings.word_embeddings.weight to text_decoder.cls.predictions.decoder.
✅ Fine-tuned model loaded from Hugging Face.
   Model      : bajughu/group22600-image-caption-generator
   Parameters: 247,444,600
```

✅ **Interactive demo**

Dataset & Preprocessing

Dataset:

COCO 2017 Caption Dataset, 600 randomly sampled images, and 3005 filtered captions

Why?

This reduced storage usage and improved processing efficiency in Google Colab.

Preprocessing Steps:

- Downloaded annotation files separately
- Created subset annotations
- Filtered captions to match images
- Verified image-caption alignment

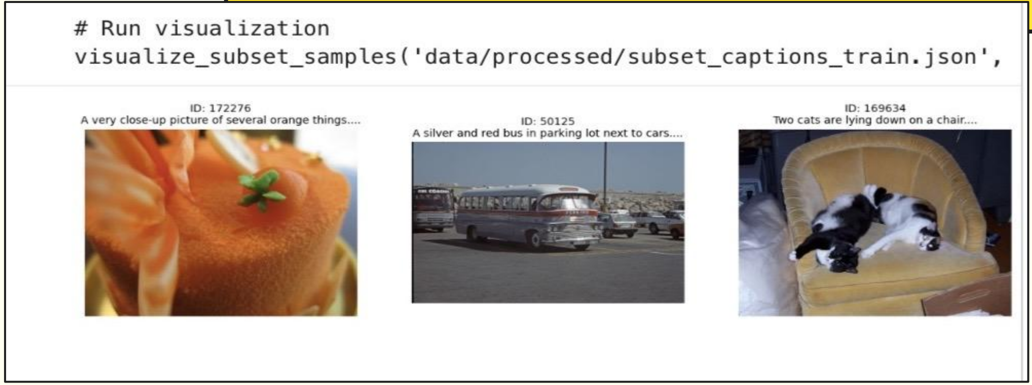
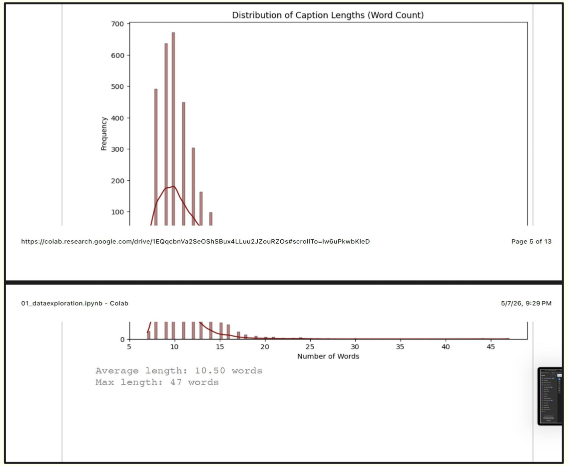


Figure 1. Sample image-caption pairs used to verify preprocessing and caption alignment.

Data Analysis:

Caption Statistics:

- Average caption length: 10.5 words
- Maximum caption length: 47 words



Diversity Verification:

Random image sampling was used to verify dataset diversity and prevent overfitting.

Why This Matters?

- Helps configure decoder settings
- Improves caption consistency
- Prevents captions from becoming too short or too long

Figure 2. Distribution of caption lengths used to analyze decoder settings and caption consistency.

Model Training:

Training Process:

- Fine-tuned BLIP captioning model
- Used Hugging Face Transformers
- Trained and tested in Google Colab

Libraries Used:

- transformers
- torch
- pycocotools
- evaluate

Objective:
Generate accurate captions that closely match human-written image descriptions.

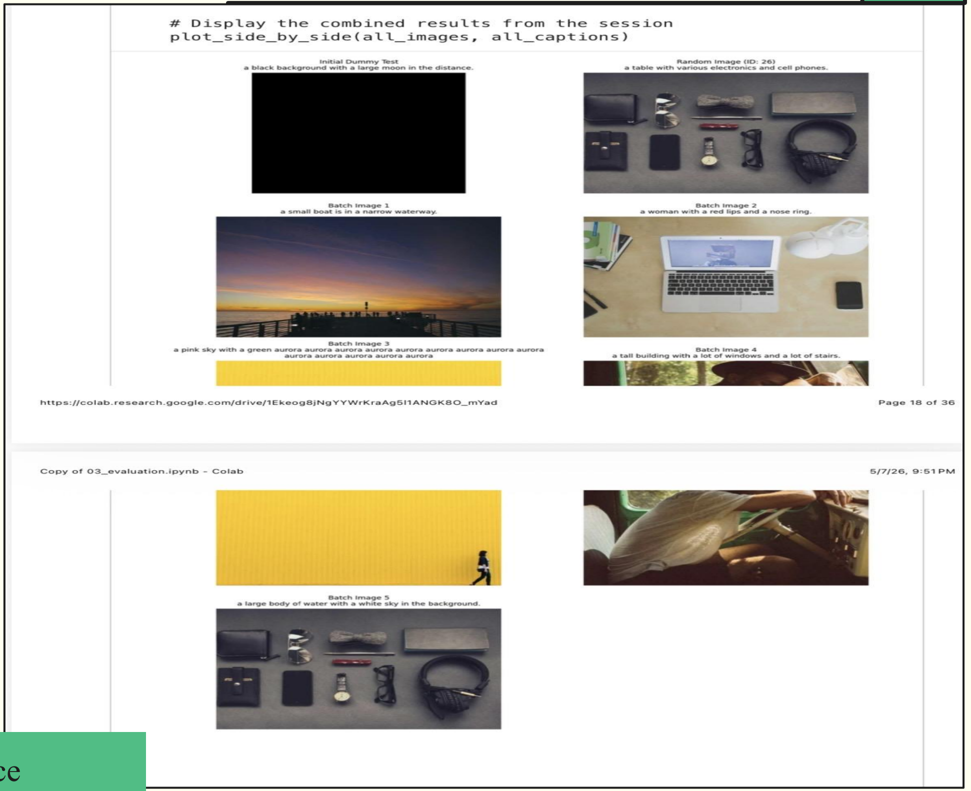


Figure 3. Multiple inference examples generated during model evaluation and testing.

Live Demo Results:

Successful Captions:

- "A photo showing a little boy riding a bike."
- "A photo showing a girl kicking a soccer ball."
- "A photo showing a baby bear in the grass."
- "A photo showing a variety of donuts and pastries."

Observations:

- Accurate caption generation
- Fast inference speed
- Worked across multiple image categories

Inference Speed:

Approximately 6–7 seconds per image in Google Colab.

Figure 5. Batch image captioning results generated by the deployed BLIP model.

Challenges and Lessons Learned:

Challenges:

- Large dataset size
- Limited Colab storage
- GPU limitations

What We Learned:

- Image preprocessing
- Hugging Face deployment
- AI model evaluation

Conclusion:

Our project successfully created an AI image caption generator that automatically describes uploaded images using deep learning.

Future Improvements

- Larger datasets
- Better captions
- Real-time caption generation

Final Takeaway:

This project showed how AI can combine computer vision and natural language processing to understand and describe images automatically.

Contributions List:

Anashrah Adil

Organized GitHub repository and project structure, Worked on data exploration and documentation, Updated README and uploaded project results.

Blessing Ajughu

Trained the BLIP image caption model, Created model training and demo notebooks, Managed dataset setup and model integration.

Wayne Lu

- Worked on evaluation notebook and model metrics
- Tested and evaluated model performance

Jackie V